

Monitoreo de alto rendimiento con BPF para servidores saturados



Jhonnatan Gil / JhonnyPong

DevOps, SRE, Observability, k8s

DevOps/SRE Engineer and Full Stack developer with experience in Java and Python languages, Chaos focus too to generate resilience for applications with this beauty practice. I like a lot the cloud especially AWS but with this role you know other services that provided other cloud providers. I like a lot the console but every move you know more tricks to do your work more efficient!

 [/in/jhonnypong/](https://in.linkedin.com/in/jhonnypong/)

El problema

El Servidor Colapsa

Las alertas de producción se disparan. Un **pico de tráfico** masivo **satura** los microservicios y la API empieza a responder con **errores** de pasarela **504** Gateway Timeout.

No hay tiempo de implementar **APM**, inyectar código de logs ni reconstruir. Cada reinicio **empeora** la sobrecarga. Necesitas ver dentro del sistema ahora mismo sin dañarlo.

Cuando un servicio falla bajo carga, **no hay tiempo** para **instrumentar** código

monitoreo tradicional

Instrumentación / instrumentalización

requieren modificar código fuente:
importar librerías, agregar
anotaciones, redeployar

cada segundo cuenta: reiniciar un
servicio para instrumentarlo puede
empeorar la crisis

equipos operan a ciegas justo
cuando más necesitan visibilidad



1. Kernel & eBPF

Captura llamadas de red y ejecución segura desde el nivel de sistema operativo.



2. Grafana Beyla

Auto-descubre procesos del espacio de usuario y los convierte en métricas OpenTelemetry.



3. Prometheus + Grafana

Almacena de forma persistente y visualiza la latencia y volumen de datos en tiempo real.

Extended Berkeley Packet Filter: programas
sandboxed que corren en el kernel Linux

eBPF

Entrañas de eBPF

Programas eBPF se adjuntan a hooks del kernel: syscalls, tracepoints, kprobes, uprobes y XDP.

Los Maps son estructuras key-value para compartir datos entre espacio kernel y usuario de forma eficiente.

Es event-driven: los programas se ejecutan solo cuando ocurre un evento, minimizando overhead a ~1-3% CPU.

El verifier del kernel garantiza seguridad: verifica que no haya loops infinitos ni accesos invalidos a memoria.

JIT compilation convierte bytecode eBPF a código nativo con rendimiento cercano al kernel nativo.

Resumen de alto nivel de eBPF

De User Space a Kernel Space

Al enganchar el programa eBPF de Beyla en estos puntos de entrada, interceptamos la duración del procesamiento y volumen sin alterar el binario de la aplicación original.

Las aplicaciones normales operan en el Espacio de Usuario. Cuando desean recibir tráfico de red, abrir sockets o escribir en el disco, hacen una transición solicitando servicios al Kernel Space mediante Syscalls.

✓ Compatible con cualquier lenguaje: Go, Rust, Java, Node.js, Python.

eBPF para observabilidad



Networking

Inspeccion de paquetes sin salir del kernel.

Parsing de protocolos custom y forwarding programable.



Tracing

Visibilidad de syscalls y funciones de aplicacion sin detener procesos. kprobes y uprobes.



Security

Filtrado de syscalls, deteccion de anomalias en tiempo real y visibilidad de red a nivel socket.



Observability

Metricas custom con agregacion en-kernel. Sin exportar samples masivos. Overhead minimo.

Instrumentación automática de aplicaciones
con eBPF y OpenTelemetry

Beyla

Que es Grafana Beyla

Herramienta **open-source** de auto-instrumentacion basada en **eBPF**, donada por Grafana Labs a OpenTelemetry (**CNCF**).

Captura metricas **RED (Rate, Errors, Duration)** y trazas distribuidas sin modificar código.

Soporta múltiples lenguajes: Go, Java, Python, Node.js, Rust, Ruby, .NET, C/C++.

Protocolos: HTTP/1, HTTP/2, HTTPS, gRPC, SQL, Redis, Kafka, Memcached.

Exporta en **Prometheus** metrics y **OpenTelemetry** traces/metrics.

Integración nativa con **Grafana Alloy** para pipelines completos de observabilidad.

Que es Grafana Beyla

OBI — OpenTelemetry eBPF
Instrumentation: el nuevo nombre del
proyecto en CNCF

100K+

Pulls mensuales Docker Hub

8+

Lenguajes soportados

CNCF

Proyecto OpenTelemetry

Metricas RED en accion

R Rate

Requests por segundo que atiende cada servicio. Detecta picos de trafico inesperados y cambios en el patron de uso.

E Errors

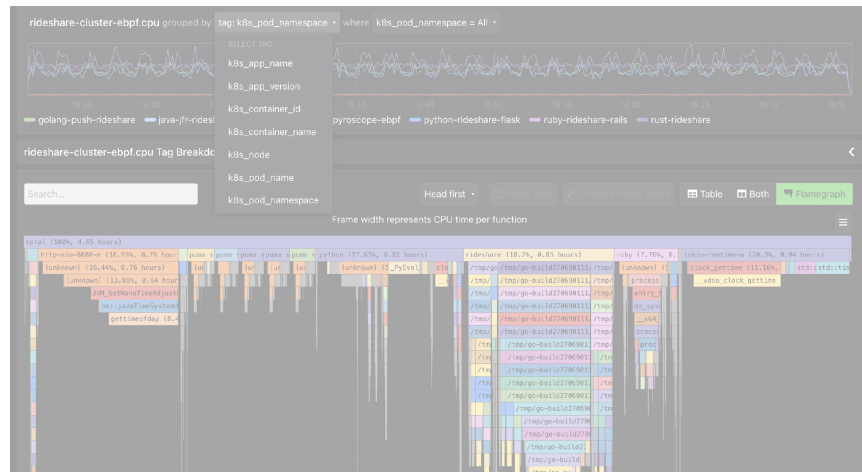
Porcentaje de requests fallidos por endpoint. Identifica degradacion de servicios y errores 5xx en tiempo real.

D Duration

Latencia p50, p90, p99 por ruta. Encuentra cuellos de botella y degradacion de latencia bajo carga.

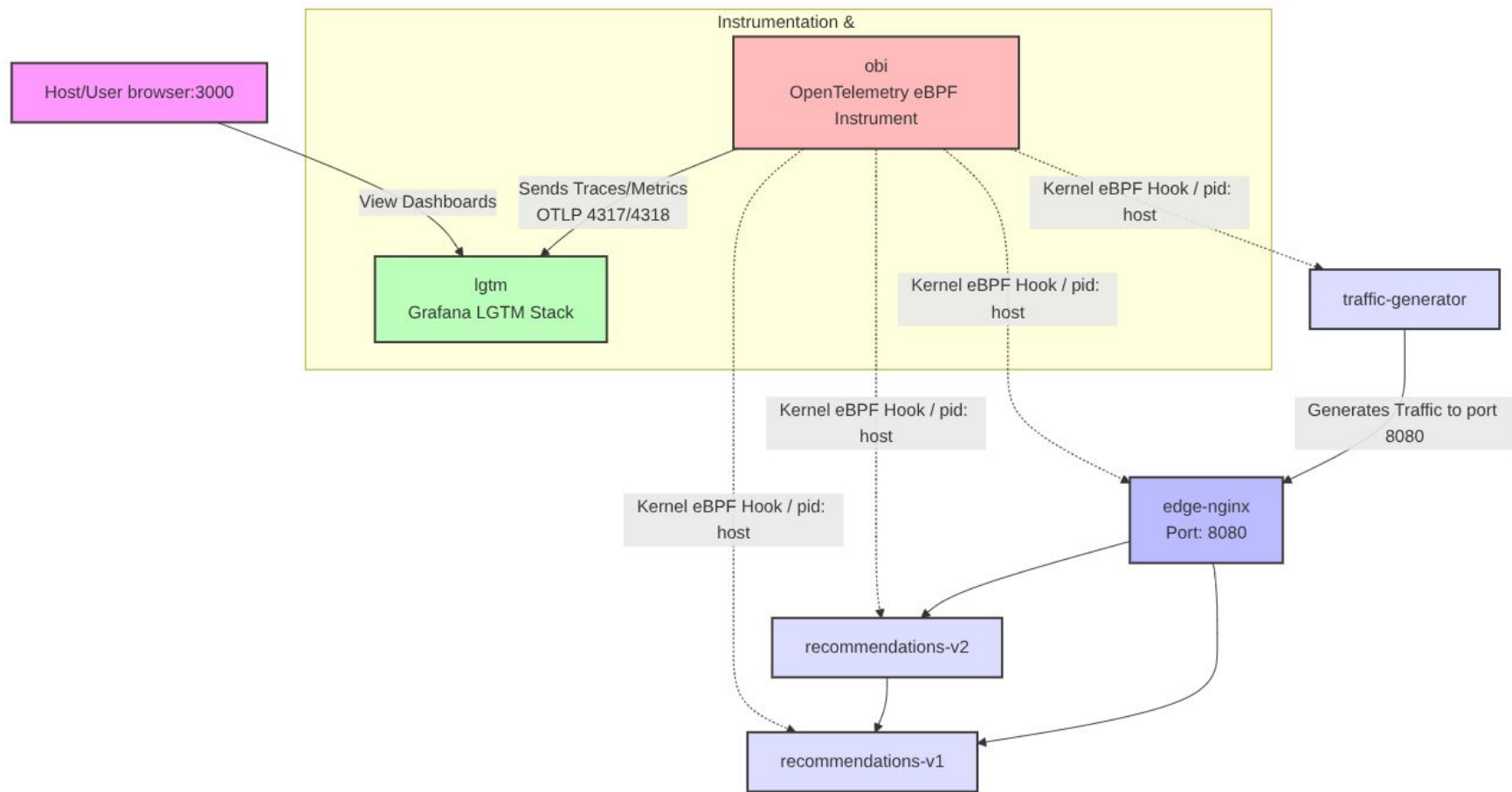
Capacidades adicionales

- Metadata Kubernetes: namespace, pod, deployment, service, node
- Service graphs: mapa automatico de dependencias entre servicios
- Alertas: umbrales en Grafana para detectar saturacion antes del colapso



o miedo?

DEMO?





<https://github.com/jthan24/openinfra-monitoreo-alto-rendimiento-ebpf>

Summary

- Instrumentacion
 - Monitoreo
 - eBPF
 - Kernel Linux
 - Grafana
 - CNCF
 - OBI
 - CNCF
 - Prometheus
 - RED metrics
-



in [/in/jhonnypong/](https://www.linkedin.com/in/jhonnypong/)

